Paper 16 — Layering Is Compensation for Dynamics: A Toy Experiment in Adaptive Depth

He Qi · with DeepSeek (AI)

Digital Life Series · Experiment Paper 6 (internal number: Paper 16)

Date: 2026-10-24

English translation of the Chinese original. The Chinese text is authoritative: https://doi.org/10.5281/zenodo.23591080

English compendium: https://doi.org/10.5281/zenodo.23592082

Series page: https://github.com/heqiq619/ai-not-alive

Abstract

The number of layers in a large model is fixed: every input goes through the entire depth. This paper asks a simple question: some inputs do not need that many layers at all, so can they take fewer? If a "layer" is a compensation for dynamics—one layer is equivalent to one step of time—then a task that needs a few steps should be given a few layers, and uniform depth is a waste.

The experiment uses an iterated task: x_{t+1} = tanh(A x_t); given an initial value x_0 and a number of steps T, predict x_T. The model is a recurrent layer with shared weights, where one layer is one step of computation. Three conditions: fixed 16 layers, adaptive (taking min(T,16) layers), fixed 6 layers (equal average compute to the adaptive one).

Results: under equal average compute (both average 6 steps), adaptive depth is better at every T—average error 1.29 versus 1.67. The largest gap is at T=1: fixed 6 layers compute a simple input six times over and actually break it (error 2.30 versus 1.49). Fixed 16 layers, meanwhile, is both expensive and worse: its compute is 2.67 times that of the adaptive one, and its average error is 1.68.

Conclusion: uniform depth loses at both ends—simple inputs are over-computed, and complex inputs are not computed enough; allocating depth on demand can trade the same compute for higher accuracy. This is consistent with the picture that "a layer ≈ a time step, and depth compensates for dynamics." Everything is toy-scale; this is not a large-model experiment.

I. The Problem

Today's large models have "uniform depth": every token passes through all layers. But inputs are not equally difficult—some sentences can be understood at a glance, while others require multi-step reasoning. If depth really corresponds to "the number of computation steps," then fixed depth makes two mistakes at once:

- Simple inputs: they go through layers they do not need, wasting compute, and may even get worse because of over-computation;

- Complex inputs: there are not enough layers, the computation cannot finish, and the error cannot come down.

What this paper measures is exactly these two things: under the same compute, is allocating depth on demand better? Is over-computation harmful?

II. Theoretical Background

This experiment comes from a picture: treat a function as its graph (a surface); evolution along the time direction "spreads out thickness" on that surface; and a frozen model has no real time evolution, so it can only spread this thickness out with depth (layers), one layer being equivalent to one step. Therefore:

- if more steps are needed ⇒ more layers are needed;

- if fewer steps are needed ⇒ fewer layers suffice, and going further is over-computation.

This picture was already written down in "Thickness and Dimension: A Geometric Picture of Residuals"; this paper turns it into a measurable experiment.

III. Experiment Design

The world: a fixed random map $x_{t+1} = \tanh(A x_t)$, where A is an 8×8 random matrix. The task is to predict x_T from an initial value x_0 and a number of steps T, with T taking the values 1, 2, 4, 8, and 16.

The model: a recurrent layer with shared weights. Each layer performs $h \leftarrow \tanh(W h + U x_0 + b)$ once, and a linear head then outputs the prediction. The number of layers = the number of steps taken, and the weights are shared across all steps.

Three conditions:

- A, fixed 16 layers: no matter what T is, it always takes the full 16 steps;

- B, adaptive: takes min(T, 16) steps;

- C, fixed 6 layers: equal average compute to B (B's average number of steps is 6).

Metrics: the average error at each T; the average number of steps (= average amount of computation).

IV. Results

In the equal-average-compute pair (B and C, both averaging 6 steps), B is better at every T:

At T=1, B is 1.49 and C is 2.30. At T=2, 1.27 versus 1.57; at T=4, 1.38 versus 1.50; at T=8, 1.22 versus 1.50; at T=16, 1.07 versus 1.46. On average, 1.29 versus 1.67.

Fixed 16 layers (A) is both the worst and the most expensive: average error 1.68, with average compute 2.67 times that of the adaptive one.

Three readings.

First, over-computation is genuinely harmful. T=1 needs only one step, yet fixed 6 layers iterate the same map six times, and the result is much worse than taking only one step. This is not "a little extra computation does not matter"; it is "extra computation breaks things."

Second, too few layers really cannot be made up for. T=8 and T=16 need more than six steps, and the error of fixed 6 layers is clearly higher.

Third, uniform depth loses at both ends: it is wasted on simple inputs, and it is not enough for complex inputs. On-demand allocation can be more accurate under the same compute.

V. Implications for Large Models

This experiment is a toy, but its shape can be carried over directly to large models:

- Not all inputs deserve the same depth. Simple tokens and simple contexts can early-exit; only what needs multi-step reasoning is worth going deep on.

- Since over-computation breaks things, "adding depth mindlessly" is not a sure gain: depth, too, must be on demand.

- In engineering this corresponds to a family of methods: early exit, per-token skipping, conditional computation. This paper proposes no new method; instead it gives this family of methods an explanation in terms of "compensating for dynamics" and a fair comparison.

Note that "on demand" has a precondition: knowing how many steps are needed. In the experiment T is given; in real settings the number of steps is estimated, and if the estimate is wrong, the gain is discounted.

VI. Limitations

First, the world is synthetic, the map is a random matrix, and the task is not language.

Second, the model is a recurrent layer with shared weights, not a transformer; layers in large models do not share weights, and the role of depth is more complex.

Third, T is given by hand; a real model does not have this prior and would need a separate "step-count estimate."

Fourth, only two quantities were measured, error and compute; engineering costs such as training stability and routing overhead were not measured.

VII. Relation to Existing Work

"Computing depth on demand" is not a new idea: there is a large body of work on early exit, recurrent depth (universal transformer), per-token layer skipping (conditional computation, mixture-of-depths), and layer pruning.

This paper does not claim that the method is original; its difference lies in two points: first, it explains the method within the picture that "a layer ≈ a time step, and depth compensates for dynamics"; second, it makes a fair comparison at equal average compute—not "our method is slightly better," but "under the same compute, on-demand allocation is better in both directions."

VIII. Reproducibility

Scripts: 850_adaptive_depth.py (main comparison), 851_adaptive_depth_v2.py (equal-compute comparison). Depends on Python 3 and numpy, no GPU, low memory footprint.

Appendix: Labels

- **[fact]** The experimental numbers (under equal compute, adaptive is better across the board; over-computation is harmful at T=1; fixed 16 layers is both expensive and worse).

- **[picture]** A layer ≈ a time step, and depth compensates for dynamics—from "Thickness and Dimension" (Paper 15), marked [conjecture].

- **[speculation]** A large model using adaptive depth can be more accurate under the same compute—an extrapolation of the toy conclusion, not verified on real models.

- **[not claimed]** No claim of a new method; both early exit and conditional computation already have prior work.

> This paper was written by He Qi, with DeepSeek (AI) assisting with experiment design and code writing; the views and responsibility belong to the author.